```
end
```

Although it doesn't have an .rb extension, the Rakefile is actually just a file of Ruby code. Rake defines an environment and methods including desc and task and then executes the Rakefile.

The desc method provides a single line of documentation for the task that follows it. The task method defines a Rake task that can be executed from the command line. The parameter is the name of the task (a symbol), and the block that follows is the code to be executed. Here we can just use rm—all the methods in FileUtils are automatically available inside Rake files.

We can invoke this task from the command line:

```
$ rake delete_unix_backups
```

One quick note that if you are using Ruby on Rails, the rails command also searches for available Rake tasks, so within a Rails application, you could execute this as rails delete_unix_backups.

Okay, now let's write a second task in the same Rakefile. This one deletes Windows backup files.

```
desc "Remove files with a .bak extension"
task :delete_windows_backups do
  files = Dir["*.bak"]
  rm(files, verbose: true) unless files.empty?
end
```

We can run this with rake delete_windows_backups.

But let's say that our application could be used on both platforms, and we wanted to let our users delete backup files on either. We *could* write a combined task, but Rake gives us a better way—it lets us *compose* tasks. Here, for example, is a new task:

```
desc "Remove Unix and Windows backup files"
task delete_backups: [:delete_unix_backups, :delete_windows_backups] do
  puts "All backups deleted"
end
```

The task's name is delete_backups, and it depends on the two other tasks. This isn't some special Rake syntax: we're simply passing the task method a Ruby hash containing a single entry whose key is the task name and whose value is the list of antecedent tasks. Rake parses the hash to create the list of task dependencies, In this case Rake will execute the two platform-specific tasks in the order they are listed before executing the delete_backups task:

```
$ rake delete_backups
rm entry~
rm index.bak list.bak
All backups deleted
```

If those dependent tasks have their own dependent tasks, then those tasks are run first, and as a result, it's possible to build complex trees of task execution while keeping each individual task relatively small.

Our current Rakefile contains some duplication between the Unix and Windows deletion tasks. As it is just Ruby code, we can define a Ruby method to eliminate this:

```
def delete(pattern)
```